# 04 Schaltnetze

04_standardcircuits, p.1

$\overline{x}$ ist “nicht x”
$\wedge$ ist UND
v ist ODER
$\oplus$ ist XOR

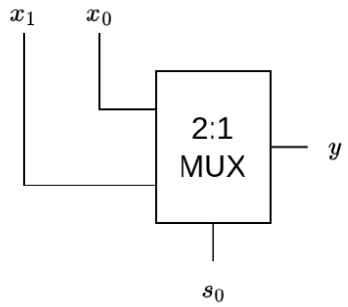
---

**Bauteile:**

1. **Multiplexer** wählen eines von mehreren Signalen aus.
2. **Addierer** rechnen, und aus ihnen entsteht auch der **Subtrahierer**.
3. **Komparator** und **Barrel-Shifter** vergleichen und verschieben.
4. Eine **ALU** ist all das zusammen, mit einem Multiplexer als Auswahlschalter.

## Multiplexer und Demultiplexer

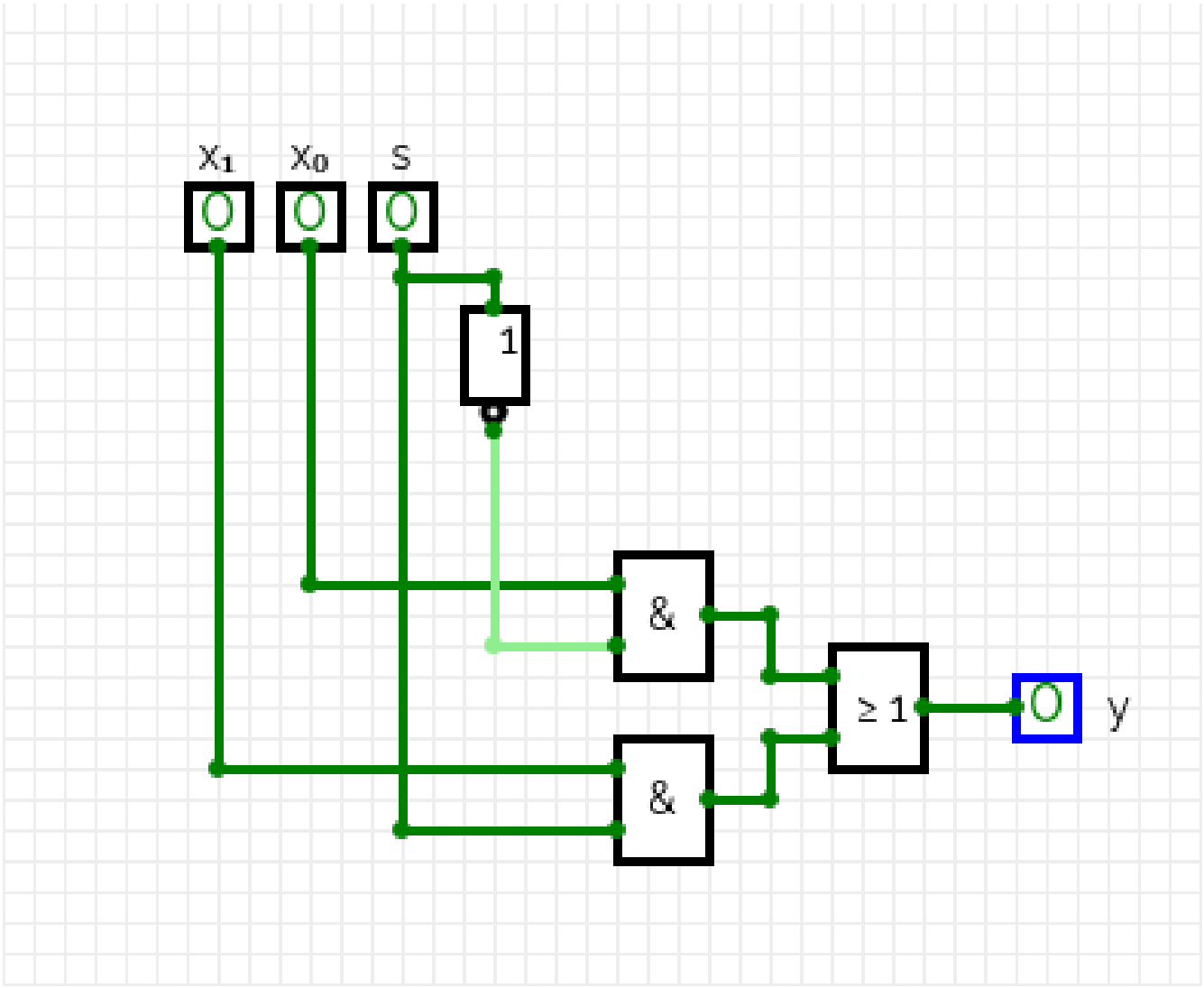
04_standardcircuits, p.7
- Ein Steuersignal entscheidet, welcher Datenpfad durchgeschaltet wird.
- mehrere Datenquellen auf eine Ausgangsleitung zusammenführen
- $Y = (x_0 \wedge \overline{s_0}) \vee (x_1 \wedge s_0)$




Schaltbild des 1-aus-2 Multiplexers
(Eingänge $x_0, x_1$, Steuersignal $s_0$, Ausgang $y$)

Wahrheitstafel

| $s_0$ | $y$ |
|---|---|
| 0 | $x_0$ |
| 1 | $x_1$ |




**4:1 MUX**

| S1 | S0 | durchgeschaltet |
|---|---|---|
| 0 | 0 | X0 |
| 0 | 1 | X1 |
| 1 | 0 | X2 |
| 1 | 1 | X3 |

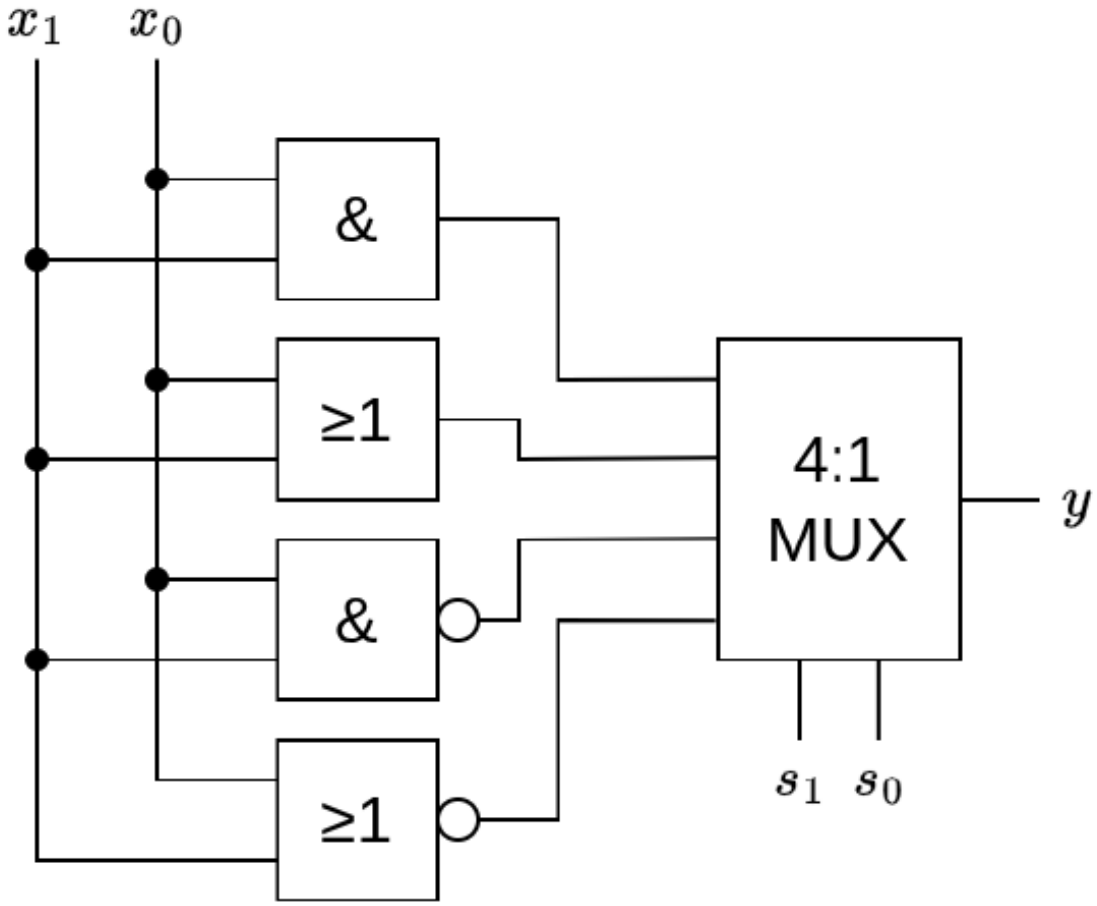
$$Y = (\overline{S_1} \wedge \overline{S_0} \wedge D_0) \vee (\overline{S_1} \wedge S_0 \wedge D_1) \vee (S_1 \wedge \overline{S_0} \wedge D_2) \vee (S_1 \wedge S_0 \wedge D_3)$$

### Multiplexer als Funktionsgenerator/Funktionsauswahl

Vorgehensweise:

1. Wahrheitstafel in obere und untere Hälfte teilen
2. $x_{n-1}$ wird zur Steuerleitung des Multiplexers, an den Eingängen stehen
   - die durch die obere Hälfte der Wahrheitstafel beschriebene Funktion (Eingang für $x_{n-1} = 0$)
   - die durch die untere Hälfte der Wahrheitstafel beschriebene Funktion (Eingang für $x_{n-1} = 1$)
3. Obige Schritte wiederholen, bis die Teiltafeln nur noch von $x_0$ abhängen

**Auswahl** zwischen verschiedenen Funktionsergebnissen
- auch fertigen Funktionen

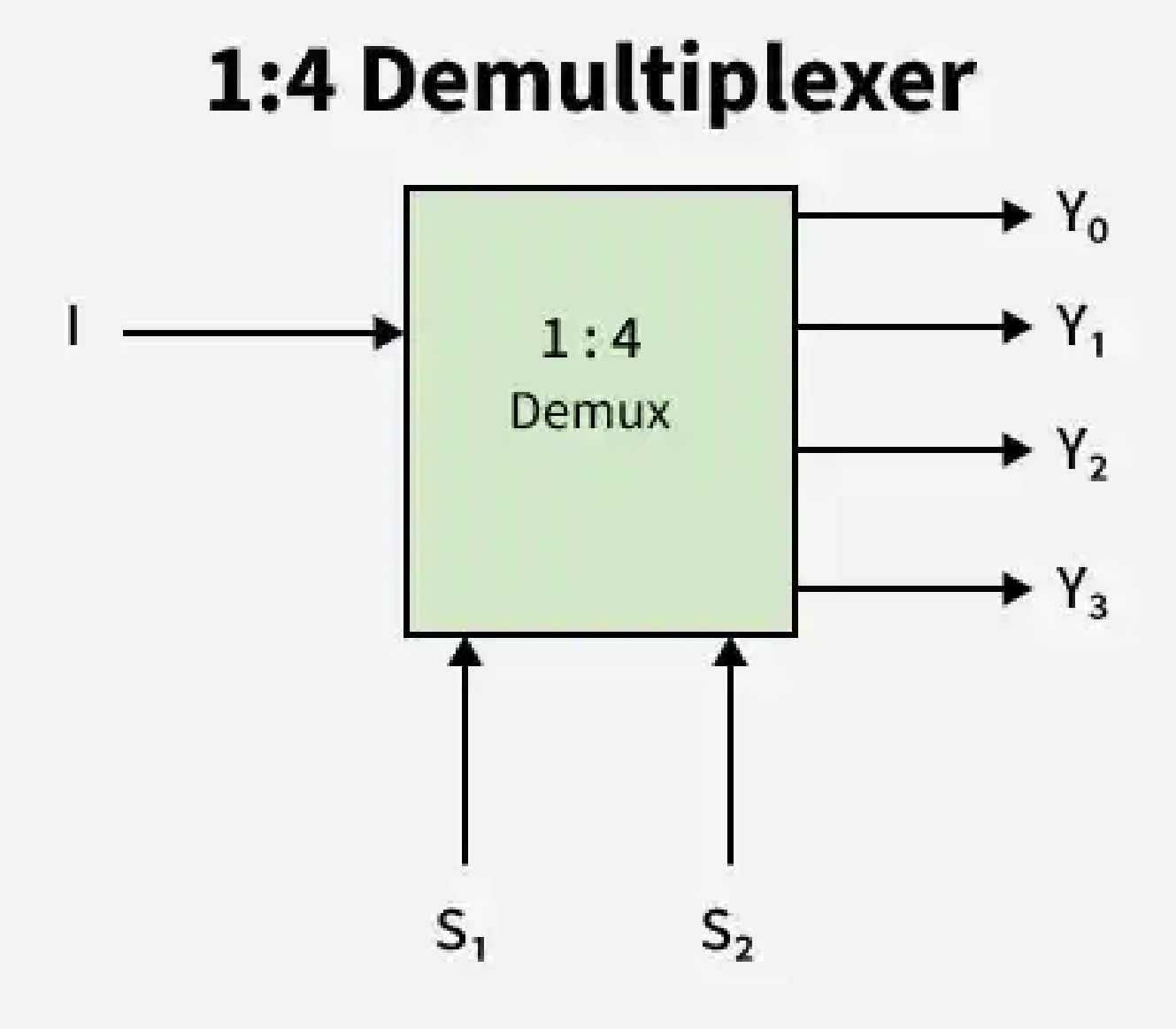



### Demultiplexer

- Umkehrung des Multiplexers
- Statt mehrere Eingänge auf einen Ausgang zusammenzuführen, schaltet ein DEMUX einen Eingang auf genau einen von mehreren Ausgängen durch
- Auch hier bestimmen die Steuerleitungen, welcher der Ausgänge mit dem einen Eingang verbunden

ist

- $y_0 = x \wedge \overline{s}$ und $y_1 = x \wedge s$




---

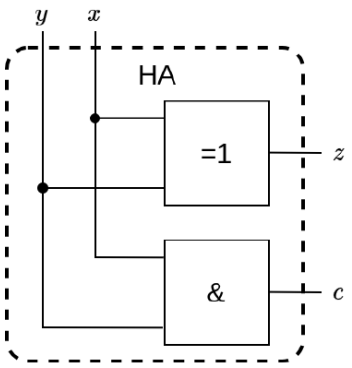
# Addierer und Subtrahierer

### Halbaddierer

1-Bit Addition von $x$ und $y$

| $x$ | $y$ | $z$ | $c$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

mit Summenbit $z$ und Übertrag $c$




Halbaddierer (HA) aus XOR- und AND-Gatter

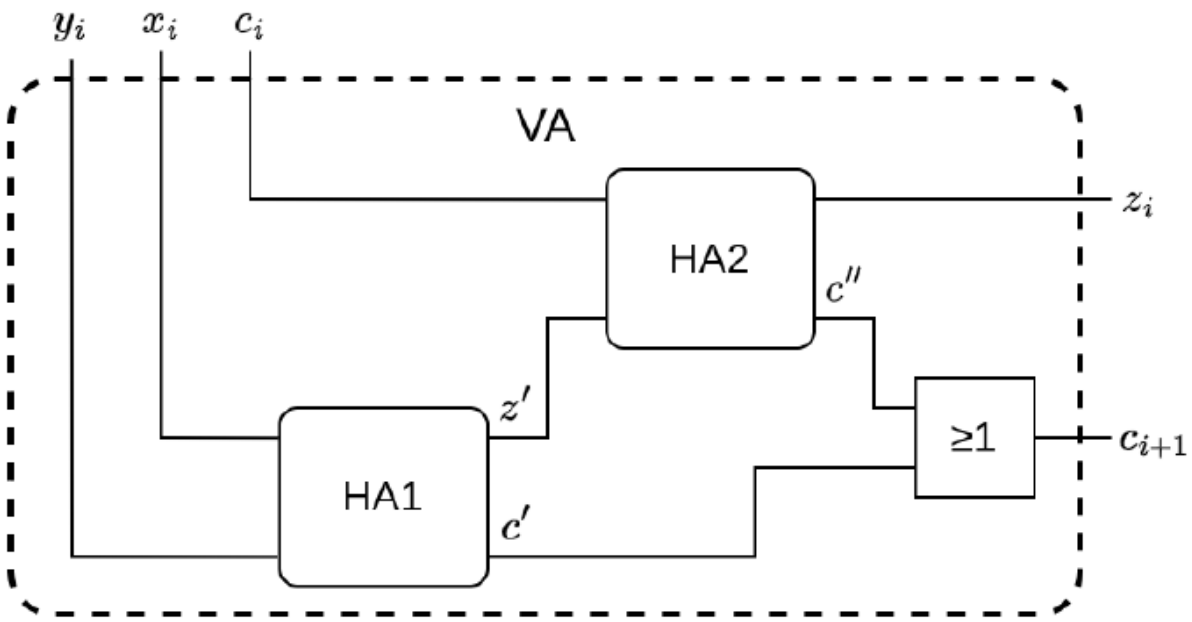
- Einfachste arithmetische Grundkomponente
- Addition zweier einzelner Bits
- x
- c: carry also der Übertrag

### Volladdierer

- Der Halbaddierer reicht nur für das niedrigste Bit.
- Ab Bit 1 muss der **Übertrag der Stelle davor** mit addiert werden.
- Der Volladdierer hat deshalb drei Eingänge (x , y , c ) und zwei Ausgänge (z , c ).
- Der Übertrag wird an den zweiten Halbaddierer übergeben

| $x_i$ | $y_i$ | $c_i$ | $z_i$ | $c_{i+1}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 |

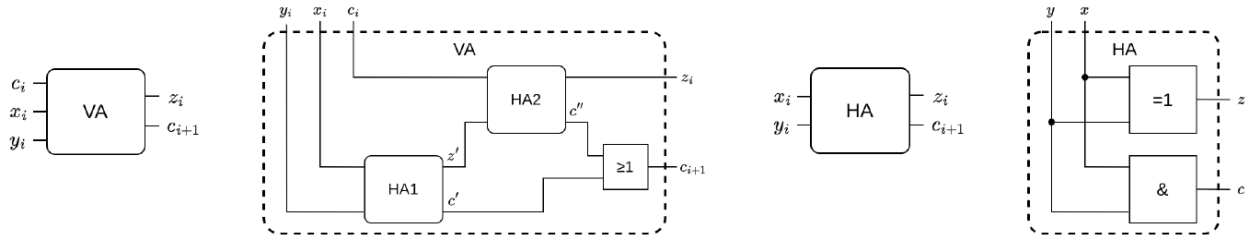
Halb und Volladdierer Bessere Erklärung




Volladdierer (VA) aus zwei HAs und ODER-Gatter

**Kapselung**

- Ersetzen des ganzen Systems durch einen neuen Block
- Nach außen zählt nur die Schnittstelle (Ein- und Ausgänge, Verhalten laut Wahrheitstafel), das Innenleben bleibt in der Black-Box.
- Wer einen VA verwendet, muss nicht mehr wissen, dass darin zwei HA stecken
- und darin wiederum ein XOR- und ein AND-Gatter

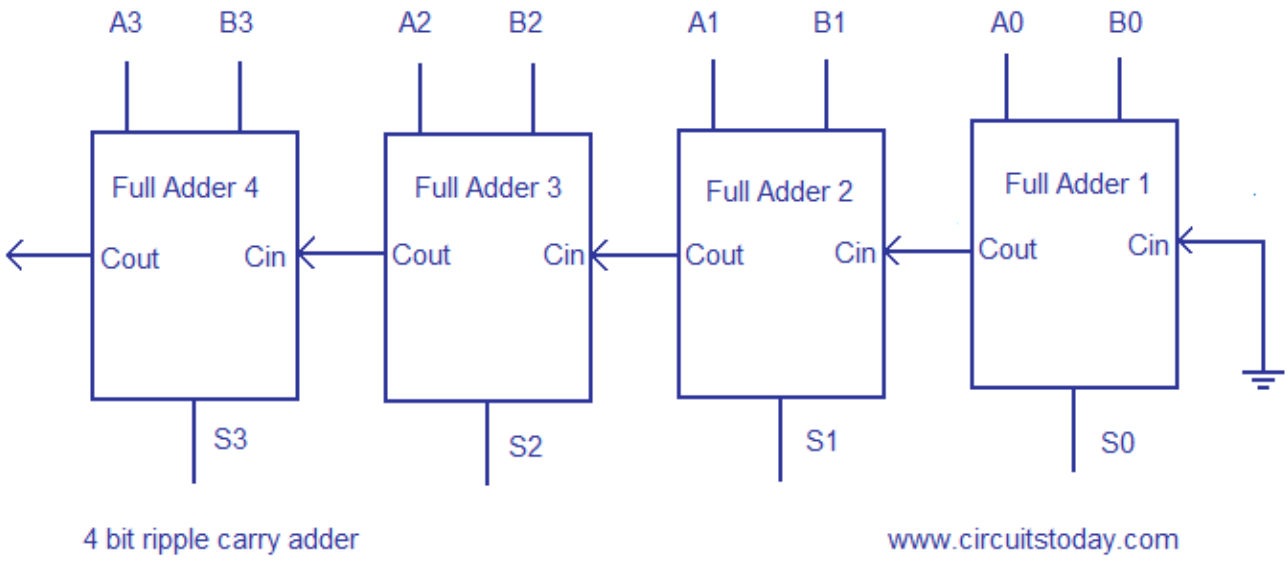



---

## Ripple-Carry-Addierer

Für n-stellige Zahlen schaltest du n Volladdierer hintereinander. Der Übertrag jedes Bausteins geht an den nächsthöheren:

| Baustein | Eingänge | Ausgänge |
|---|---|---|
| VA[0] | $(x_0, y_0, c_0 = 0)$ | $(z_0, c_1)$ |
| VA[1] | $(x_1, y_1, c_1)$ | $(z_1, c_2)$ |
| VA[2] | $(x_2, y_2, c_2)$ | $(z_2, c_3)$ |
| VA[3] | $(x_3, y_3, c_3)$ | $(z_3, c_4)$ |
| ... | ... | ... |




---

## Subtraktion

### mit dem Zweierkomplement

- Die Subtraktion (a - b) lässt sich als Addition einer negativen Zahl interpretieren:
- (a - b = a + (-b))
- mit (-b) als Zweierkomplement von (b)
- **Beispiel:** Gegeben ist (a = 2), (b = 3), gesucht ist (a - b = -1)

```
  0011   (b=3)
  1100   Einerkomplement (jedes Bit negieren)
+    1
  1101   (-b=-3) im Zweierkomplement

  0010   (a=2)
+ 1101   (-b=-3)
  1111

  1111
  0000
```

```
+    1
  0001 -> 1
a + b = -1
```

---

## Weitere Standardschaltnetze

### Komparator

- Vergleichen von Werten auf größe
- zwei Eingänge
- drei Ausgänge

Wahrheitstafel für 1-Bit Komparator

| $x$ | $y$ | $x > y$ | $x = y$ | $x < y$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 |

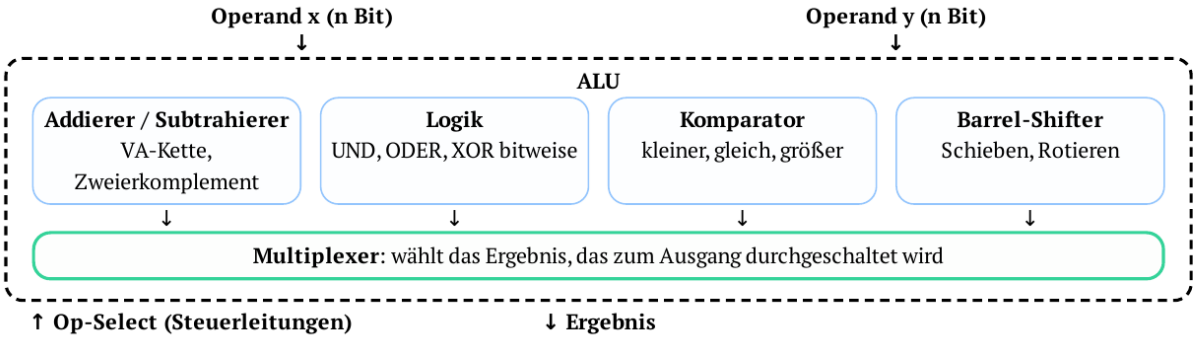
### Barrel-Shifter

Er verschiebt Bitmuster **in einem Schritt**, egal um wie viele Stellen. Wichtig ist der Unterschied zwischen Schieben und Rotieren.
??

---

## Arithmetisch Logische Einheit

- soll addieren, subtrahieren, bitweise verknüpfen, vergleichen und schieben




**Die ALU ist kein neuer Baustein, sondern die Bündelung von weiteren Standardschaltnetze.**